MIIA

Maestría en Inteligencia Analítica para la Toma de Decisiones (Analytics)

SESIÓN 10

Deep Learning

Redes Neuronales Recurrentes y Predicciones Secuenciales

Daniel F. Villarraga

2026-02

Lo que cubriremos hoy

Al final de esta sesión serán capaces de:

1. Identificar áreas de aplicación del análisis de series de tiempo.
2. Distinguir las distintas aplicaciones del análisis de series de tiempo.
3. Realizar pronósticos basados en la estacionalidad y la autocorrelación.
4. Realizar pronósticos con modelos SARIMA.
5. Describir los componentes de una RNN y el cálculo de sus predicciones y estado oculto.
6. Implementar redes neuronales recurrentes en PyTorch.

¿En dónde encontrarlas?

Finanzas
Precio de una acción
$102 \rightarrow 105 \rightarrow 101 \rightarrow 108$

Clima
Temperatura por hora
$18° \rightarrow 19° \rightarrow 21°$

Lenguaje
Palabras en una frase
La $\rightarrow$ lluvia $\rightarrow$ llegó

Comercio minorista
Ventas por hora
$12 \rightarrow 18 \rightarrow 15 \rightarrow 24$ unid./h

Tráfico
Vehículos por minuto
$120 \rightarrow 135 \rightarrow 98$ veh/min

Sensores e industria
Vibración de una máquina
$0.2 \rightarrow 0.4 \rightarrow 0.9$ mm/s

Una secuencia: datos ordenados en el tiempo o por pasos.

Aplicaciones de Modelado de Secuencias

Tres objetivos comunes del modelado de secuencias:

Imputación

Completar observaciones faltantes.

Sensor (°C):
$18 \rightarrow 19 \rightarrow ? \rightarrow 22$

Pronóstico

Predecir valores futuros.

Ventas (unid./h):
$12 \rightarrow 18 \rightarrow 15 \rightarrow ?$

Clasificación

Asignar una etiqueta a la secuencia.

ECG: normal / arritmia

Pronóstico en series de tiempo

A partir de los datos observados hasta t, estimamos valores futuros:



Datos CTA para pronosticar abordajes

¿Qué contiene el dataset?

Serie diaria del sistema de transporte de Chicago (CTA), desde 2001: abordajes en buses y trenes.

- `service_date`: fecha del servicio.
- `day_type`: W (laborable), A (sábado), U (domingo/festivo).
- `bus`, `rail_boardings`, `total_rides`: abordajes por bus, tren y total.

Descargar el CSV desde el portal oficial de Chicago.

Cargar en Python con pandas

Descarga el archivo y guárdalo como `cta_ridership_daily.csv`:

```
import pandas as pd

df = pd.read_csv(
    "cta_ridership_daily.csv",
    parse_dates=["service_date"],
    thousands = ","
)
df.head()
```

Lags y autocorrelación

Explora `total_rides` en lags de 1, 7, ≈ 30 y ≈ 365 días. ¿Cuáles podrían reflejar estacionalidad?

Diferencias entre días consecutivos

`diff()` calcula $\Delta y_t = y_t - y_{t-1}$. Compara su autocorrelación con la serie original.

```
df["total_rides"].diff()
```

Grafica la autocorrelación

```
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

plot_acf(df["bus"], lags=31)
plt.show()
```

Naive Forecasting guiado por la ACF

Usa los patrones de autocorrelación para elegir qué valor pasado reutilizar:

Persistencia: autocorrelación alta en lag 1

El valor más reciente sirve como pronóstico base.

$$\hat{y}_{T+h} = y_T$$

Estacionalidad: picos cada s lags

Repite el último valor de la misma fase del ciclo. Si la ACF muestra picos en 7, 14, 21, ..., prueba $s = 7$: para CTA diario, usa el mismo día de la semana anterior.

$$\hat{y}_{T+1} = y_{T+1-s}$$

Comprueba la línea base en un conjunto de validación temporal (por ejemplo, con MAE). La ACF sugiere lags; la validación mide si pronostican bien.

SARIMA: formulación y modelo

Forma aditiva simplificada (ciclo semanal, $s = 7$). SARIMA$(1,0,1)X(2,1,0)_7$

$$
\begin{aligned}
W_t &= y_t - y_{t-7} \\
W_t &= \mu + \phi_1 W_{t-1} + \phi_2 W_{t-7} + \phi_3 W_{t-14} \\
&\quad + \theta_1 \varepsilon_{t-1} + \varepsilon_t
\end{aligned}
$$

W_t: serie con diferencia estacional; los lags 7 y 14 representan una y dos semanas.

```
from statsmodels.tsa.arima.model import ARIMA

modelo = ARIMA(
    serie, order=(1, 0, 1),
    seasonal_order=(2, 1, 0, 7)
)
ajuste = modelo.fit()
```

Preparación de Datos para Redes Neuronales

Se deben crear ventanas de datos con 56 observaciones y un objetivo.

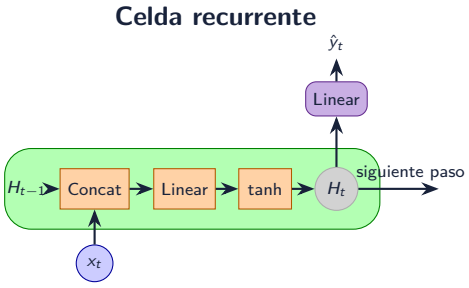
```
import torch

class TimeSeriesDataset(torch.utils.data.Dataset):
    def __init__(self, series, window_length):
        self.series = series
        self.window_length = window_length

    def __len__(self):
        return len(self.series) - self.window_length

    def __getitem__(self, idx):
        if idx >= len(self):
            raise IndexError("dataset index out of range")
        end = idx + self.window_length  # 1st index after window
        window = self.series[idx : end]
        target = self.series[end]
        return window, target
```

Redes Neuronales Recurrentes

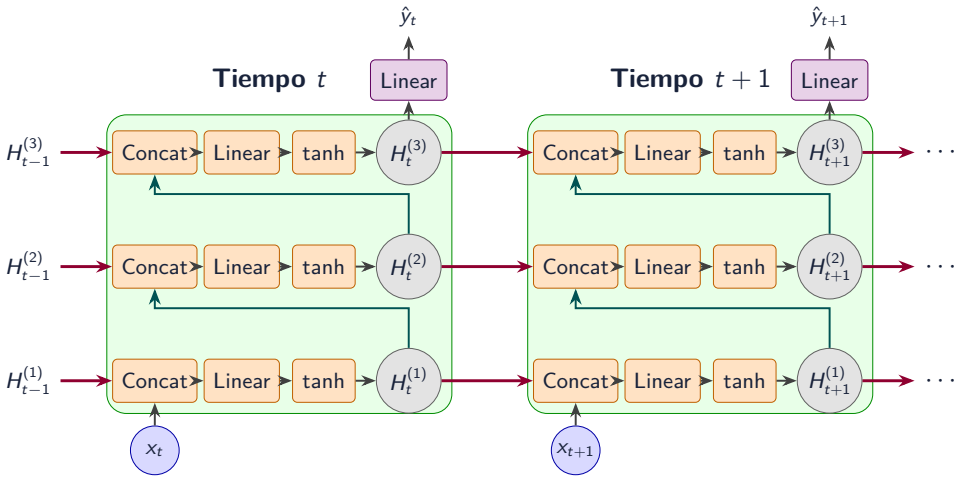


Los parametros se comparten. La celda t y $t+1$ tienen los mismos parametros.

```python
import torch
from torch import nn

class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.memory = nn.Sequential(
            nn.Linear(input_size + hidden_size, hidden_size),
            nn.Tanh()
        )
        self.output = nn.Linear(hidden_size, output_size)
    def forward(self, x):
        batch_size, window_length, dimensionality = x.shape
        x_time_first = x.transpose(0, 1)
        H = torch.zeros(batch_size, self.hidden_size, device=x.device)
        for x_t in x_time_first:
            H = self.memory(torch.cat([x_t, H], dim=1))
        return self.output(H)
```

RNNs Profundas



RNN Profunda en Pytorch

```
from torch import nn

class SimpleAbstractDeepRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers=3, batch_first=True)
        self.out = nn.Linear(hidden_size, output_size)
    def forward(self, x):
        output, hidden = self.rnn(x)
        return self.out(output[:, -1])
```

Actividad en Grupo

En equipos, pronostiquen la demanda diaria con el conjunto Bike Sharing de UCI.
Datos: usen `day.csv`.
Procedimiento:

1. Reserven el último 20 % del tiempo para prueba; no mezclen las fechas.
2. Comparen tres pronósticos a un día: naïve (persistencia y, si la ACF lo sugiere, lag 7), SARIMA y RNN con ventana de 14 días.
3. Evalúen los tres modelos en el mismo período usando MAE.

Entregable: notebook(.ipynb) con todo el analisis, gráfico de pronósticos, tabla de MAE y una breve conclusión.